UNIVERSIDAD
DE GRANADA

Metaheurísticas

Práctica 2
Planificación de Exámenes Universitarios mediante Algoritmos Genéticos

Ilias Ahmed Ahmed

16–04–2026

Índice

1. Introducción **2**

2. Modelado del problema y Representación **2**
2.1. Representación de la solución (El Cromosoma) 2
2.2. Función objetivo y penalizaciones 3

3. Diseño y decisiones del Algoritmo Genético **3**
3.1. Población Inicial 3
3.2. Selección 3
3.3. Cruce: ($P_c = 0{,}8$) 4
3.4. Mutación ($P_m = 0{,}4$) 4
3.5. Elitismo 4

4. Implementación e Integración del Código **4**

5. Resultados y Comparativa **6**

6. Análisis Visual de las Gráficas **7**
6.1. Curvas de Convergencia con cada Instancia 8
6.2. Comparativa de Costes y Tiempos 9

7. Análisis del Impacto del Criterio de Parada **9**
7.1. Justificación de la Mejora 10
7.2. Tamaño de la Población (N) 10
7.3. Probabilidad de Cruce (P_c) 11
7.4. Probabilidad de Mutación (P_m) 12

8. Conclusiones Finales **12**

9. Repositorio del Código **13**

1. Introducción

En esta segunda práctica he continuado trabajando con el problema de la planificación de exámenes universitarios, pero dando un salto de calidad en el enfoque de resolución ya que, si en la práctica anterior utilizábamos estrategias de búsqueda basadas en trayectorias (moviendo y modificando una única solución paso a paso), en esta práctica he implementado como se nos pide en el enunciado, una metaheurística basada en poblaciones, un **Algoritmo Genético**.

El objetivo principal de esta práctica no es solo programar el algoritmo, sino evaluar sus resultados directamente contra mis implementaciones del Primer Mejor y del Mejor Vecino de la Práctica 1. La idea inicial de la que partía era que, al imitar la selección natural y mantener un conjunto de soluciones (población) que evolucionan a la vez intercambiando información y explorando nuevas variantes (mutación), el Genético sería capaz de solventar el hecho de quedarse atascado en los óptimos locales como las búsquedas locales, especialmente cuando se trabajaba con las instancias de mayor tamaño. Pero no sabía realmente en tema de eficiencia, cuánto tardaría con respecto a los demás.

Para garantizar que esta comparativa fuera cien por cien justa y empírica, he mantenido intactas las reglas de evaluación: he reutilizado el mismo generador de instancias (`generaInstancias.py`) con la misma semilla (`seed=42`) y he aplicado exactamente la misma función objetivo matemática. De esta forma, cualquier diferencia en el coste final o en los tiempos de ejecución recae única y exclusivamente en la eficiencia de los algoritmos utilizados y además los resultados de los algoritmos de búsqueda local de la práctica anterior mantienen los mismos resultados, así permitiendome compararlos de manera justa.

2. Modelado del problema y Representación

2.1. Representación de la solución (El Cromosoma)

En los Algoritmos Genéticos se habla de representar a los individuos mediante un vector de genes. Sin embargo, tras analizarlo, decidí que la representación más eficiente en mi caso seguía siendo el diccionario de Python que diseñé en la práctica anterior: `examen ->(slot, aula)`

He tomado la decisión de mantener esta estructura como mi 'cromosoma' por tres motivos principales para el rendimiento de los algoritmos:

- **Acceso instantáneo $O(1)$:** En un Algoritmo Genético, si tengo una población de 60 individuos y ejecuto decenas de generaciones, voy a tener que evaluar la función objetivo miles de veces. Usar un diccionario hace que consultar qué alumno va a qué examen sea rapidísimo en memoria, porque en caso contrario tardaría demasiado como para estar haciendo pruebas de lo que voy desarrollando.

- **Garantía de integridad:** Al ser el ID del examen la clave del diccionario, es matemáticamente imposible que un mismo examen se asigne a dos franjas horarias distintas a la vez. El propio diseño de la estructura cumple la tercera restricción dura del problema automáticamente.

- **Facilidad para el cruce:** Para intercambiar material genético entre dos padres, solo tengo que recorrer las claves del diccionario de uno de ellos y decidir qué valor hereda el hijo, por lo que se me hace más sencillo trabajar con el diccionario.

2.2. Función objetivo y penalizaciones

Para evaluar la calidad (**fitness**) de cada individuo, he reciclado mi archivo `funcionObjetivo.py`. La formulación sigue transformando el problema en un modelo de minimización de costes, donde un coste menor indica un individuo más apto para sobrevivir, ya que tiene mejores características que otro con mayor coste y por ende, generará mejores hijos para las siguientes generaciones.

$$f(x) = 1 \cdot P_{consecutivos} + 2 \cdot P_{mismo_dia} + 1 \cdot P_{distribucion} + 100000 \cdot V$$

He mantenido el peso desproporcionado (100,000) para los no cumplimientos de restricciones duras (V), como solapamientos de estudiantes o excesos de aforo. Esto es fundamental en el Genético para asegurar que, durante la selección, los individuos que representen horarios legales sean mucho mejores que los que no lo son y por tanto sean elegidos para formar las siguientes generaciones.

3. Diseño y decisiones del Algoritmo Genético

Para que el algoritmo funcionara correctamente sin quedarse atascado ni desviarse del objetivo, tuve que tomar varias decisiones en el diseño para cada uno de los operadores genéticos tras realizar varias pruebas empíricas. He optado por un **Algoritmo Genético Generacional con Elitismo**, ajustado con los siguientes parámetros:

3.1. Población Inicial

Inicialmente, me planteé crear la primera población de forma totalmente aleatoria. Sin embargo, al hacer pruebas, me di cuenta de que el algoritmo perdía demasiadas generaciones simplemente intentando encontrar horarios que no superasen el aforo de las aulas.

Para solucionar esto y darle un empujón inicial, decidí reutilizar mi función `solucion_inicial()` de la Práctica 1. En la que genero los 60 individuos de mi población inicial asignando clases aleatorias pero con un tope de intentos para asegurar que, al menos de base, el aforo se respete. Esto acelera muchísimo la convergencia en las primeras etapas.

3.2. Selección

A la hora de elegir qué individuos se reproducen, descarté rápidamente el método de la Ruleta. Porque al tener penalizaciones de 100,000, la varianza del fitness es muy alta. Si un individuo tiene 2 solapamientos y el resto de la población tiene 10, en una ruleta el primer individuo se llevaría casi toda la probabilidad de selección, provocando una convergencia demasiado rápida (toda la población se volvería idéntica a él en dos generaciones).

En cambio, implementé una selección por Torneo con $k = 5$. Es decir, en cada torneo, selecciono 5 individuos al azar de la población y me quedo con el que tenga el menor coste para usar parte de sus genes en la siguiente generación. He elegido un $k = 5$ (en lugar de lo que se suele usar $k = 2$ o 3) para aumentar el abanico de individuos que compiten y tener más posibilidades de elegir al mejor de la generación. Así me aseguro de que los individuos mediocres se descarten más rápido y los mejores genes se heredan en las siguientes poblaciones hasta llegar a una buena solución final.

3.3. Cruce: ($P_c = 0{,}8$)

El operador de cruce se encarga de generar nuevos individuos combinando soluciones buenas. Como en mi caso tengo el cromosoma en forma de diccionario (donde el orden de los exámenes realmente no importa), un cruce de un punto o dos partes no creo que tuviese mucho sentido.

Por ello, decidí usar un Cruce Uniforme, en el que recorro todos los exámenes del problema, "tiro una moneda"(50 % de probabilidad) y el Hijo 1 hereda la configuración del Padre 1 o del Padre 2. El Hijo 2 se queda con el inverso. Le asigné una probabilidad de aplicación bastante alta (80 %) para asegurarme de que haya una alta herencia de material genético de generación en generación y así no haya cambios tan bruscos.

3.4. Mutación ($P_m = 0{,}4$)

La mutación es lo que me asegura no perder diversidad genética. Normalmente se usan probabilidades muy bajas, pero en mi implementación he decidido usar un $P_m = 40\,\%$, ya que he ido probando diferentes opciones y esta ha sido mejor que los valores bajos como 0.1. La justificación técnica es que mi función de mutación es muy conservadora: si el individuo muta, solo cambia un único examen al azar (asignándole nueva aula y franja). En problemas de hasta 200 exámenes, alterar solo 1 de ellos es un cambio mínimo, por lo que necesitaba una alta probabilidad del 40 % para garantizar que la población en su conjunto experimente la suficiente exploración para escapar de óptimos locales y poder avanzar algo más de lo que lo hacía con una mutación de 0.1.

3.5. Elitismo

Finalmente, para evitar el efecto destructivo que a veces tienen el cruce y la mutación, implementé un Elitismo simple de $E = 1$. Antes de empezar a generar la nueva población, localizo al individuo con el coste más bajo de la generación actual y lo copio directamente y sin alteraciones a la siguiente generación. Esto me garantiza matemáticamente que el mejor coste histórico de mi programa nunca va a empeorar y las generaciones van a mejor.

4. Implementación e Integración del Código

Para el desarrollo del proyecto, he seguido la misma arquitectura modular que he usado en la práctica 1. He copiado mis archivos `generaInstancias.py`, `funcionObjetivo.py`,

`inicial.py` y `busquedaLocal.py` de la P1, ya que funcionan perfectamente y me sirven para evaluar a los individuos y para ejecutar la comparativa de una manera justa como mencioné previamente, así obteniendo resultados más fiables.

La lógica del nuevo algoritmo que hay que añadir está en `algoritmoGenetico.py`, donde he encapsulado todas las funciones descritas anteriormente.

```python
def algoritmo_genetico(exams, rooms, mapa_estud, n_slots, n_pob=60, n_gen=200, max_sin_mejora=20, prob_cruce=0.8, prob_mutacion=0.4):
    pob = [solucion_inicial(exams, rooms, n_slots) for _ in range(n_pob)]
    historial_evolucion = []

    mejor_historico_coste = float('inf')
    generaciones_sin_mejora = 0

    for gen in range(n_gen):
        pob_evaluada = [(ind, evaluar_detallado(ind, exams, rooms, mapa_estud, n_slots)[0]) for ind in pob]
        pob_evaluada.sort(key=lambda x: x[1])

        mejor_actual = pob_evaluada[0]
        historial_evolucion.append(mejor_actual[1])

        if mejor_actual[1] < mejor_historico_coste:
            mejor_historico_coste = mejor_actual[1]
            mejor_actual_completo = mejor_actual[0].copy()
            generaciones_sin_mejora = 0
        else:
            generaciones_sin_mejora += 1

        if generaciones_sin_mejora >= max_sin_mejora:
            break

        nueva_pob = [mejor_actual[0].copy()]

        while len(nueva_pob) < n_pob:
            padre1 = seleccion_torneo(pob_evaluada, k=5)
            padre2 = seleccion_torneo(pob_evaluada, k=5)

            if random.random() < prob_cruce:
                hijo1, hijo2 = cruce_uniforme(padre1, padre2)
            else:
                hijo1, hijo2 = padre1.copy(), padre2.copy()

            nueva_pob.append(mutacion(hijo1, exams, rooms, n_slots, prob_mutacion))
            if len(nueva_pob) < n_pob:
                nueva_pob.append(mutacion(hijo2, exams, rooms, n_slots, prob_mutacion))

        pob = nueva_pob

    return mejor_actual_completo, mejor_historico_coste,
```

```
    historial_evolucion
```

Listing 1: Bucle principal del Algoritmo Genético optimizado

Me aseguré de usar el método '.copy()' adecuadamente al guardar al élite de cada generación y al gestionar los padres para que los hijos fuesen entidades totalmente independientes en memoria.

5. Resultados y Comparativa

Para evaluar la efectividad real de los algoritmos que he desarrollado, he configurado el archivo principal (`main.py`) para que ejecute los tres algoritmos (Primer Mejor, Mejor Vecino y Genético) consecutivamente con las tres mismas instancias de diferentes tamaños (Pequeña, Media y Grande).

Tras afinar el modelo empíricamente, es decir, probando diferentes parámetros para ver cómo influyen en los resultados, establecí los parámetros de Población = 60 y Generaciones = 200. Aunque esto le da al genético un margen amplio de evaluaciones, se compensa perfectamente gracias al criterio de parada temprana que detiene la ejecución si detecta estancamiento en las evaluaciones.

Estos son los excelentes datos que ha devuelto la ejecución final optimizada:

Algoritmo	**Coste Final Obtenido**	**Tiempo CPU (s)**
Instancia Pequeña (50 Exámenes)		
Primer Mejor	7.603.153,08	7,78
Mejor Vecino	15.603.290,56	18,65
Algoritmo Genético	**8.503.194,08**	**30,56**
Instancia Media (100 Exámenes)		
Primer Mejor	8.704.058,30	34,96
Mejor Vecino	22.704.264,95	37,64
Algoritmo Genético	**10.404.128,30**	**90,47**
Instancia Grande (200 Exámenes)		
Primer Mejor	1.402.980,58	75,04
Mejor Vecino	20.203.681,20	74,34
Algoritmo Genético	**3.603.076,46**	**181,15**

Cuadro 1: Resultados con la configuración Genética Optimizada ($P_m = 0{,}4, N = 60$).

Observando estos resultados, hay un par de conclusiones bastante claras que se pueden apreciar. Lo primero que me llamó la atención (y que al principio me frustró un poco) es que

el Primer Mejor sigue siendo el mejor indiscutible, ganando tanto en coste como en tiempo en las tres instancias. Pero, analizándolo bien, tiene mucho sentido, ya que al aceptar el primer cambio positivo que encuentra, avanza rapidísimo. No pierde el tiempo mirando otras opciones y eso le permite dar muchísimos pasos seguidos y explicar muchos vecindarios, profundizando en un mínimo local de muy buena calidad en poco tiempo. En el extremo opuesto tenemos el Mejor Vecino, que es un bastante peor, sobre todo en la instancia grande (se queda en más de 20 millones de coste). Su problema es que, como se obliga a evaluar absolutamente todo el vecindario antes de mover un solo examen, gasta todo su tiempo de CPU para dar un solo paso, quedándose atascado casi al instante en el primer mínimo local que se cruza aunque no sea bueno. Y aquí es exactamente donde el Algoritmo Genético demuestra por qué lo he implementado. Es cierto que es el algoritmo más lento (181 segundos en la instancia grande), pero es algo lógico y esperado, porque el programa tiene que mantener, cruzar, mutar y evaluar 60 diccionarios enteros en cada iteración, aunque son diccionarios pequeños conllevan tu tiempo de evaluación. Sin embargo, compensa este esfuerzo computacional con la calidad de sus soluciones. Gracias a que mantiene una población variada y a la alta probabilidad de mutación que le configuré, el algoritmo genético consigue saltar esos mínimos locales donde quedaba atrapado el Mejor Vecino. Bajar de 20 millones a 3,6 millones en el problema más difícil me demuestra empíricamente que la exploración simultánea a través de la evolución funciona muy bien y más que lo podría hacer mejorando los parámetros, pero no lo he hecho para poder probarlos bastante y compararlos de manera justa.

6. Análisis Visual de las Gráficas

La configuración ajustada del Algoritmo Genético ha demostrado ser bastante potente, mejorando por completo los resultados obtenidos por la Búsqueda de Mejor Vecino y quedándose extremadamente cerca del algoritmo del Primer Mejor.

El siguiente paso que he seguido para entender el por qué de estos números, ha sido analizar la dinámica de convergencia media gráficas que he generado para representar mejor los datos recopilados.

6.1. Curvas de Convergencia con cada Instancia

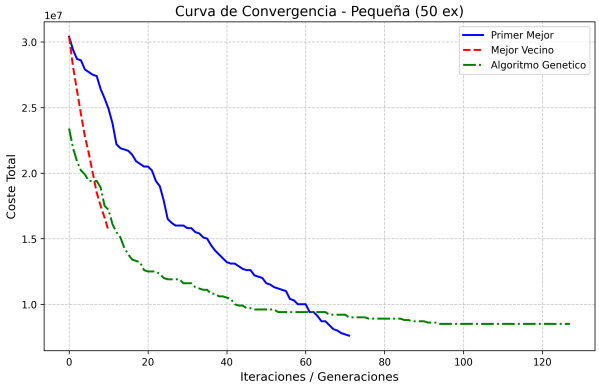

Figura 1: Instancia Pequeña.

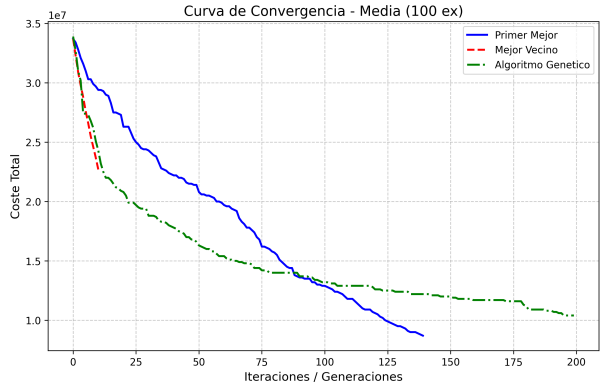

Figura 2: Instancia Media.

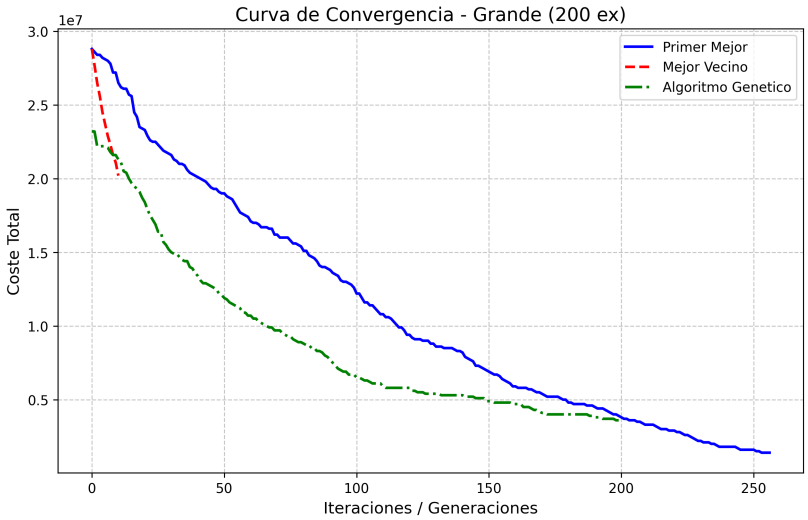

Figura 3: Instancia Grande. Con el brutal descenso de la curva Verde.

Al observar detalladamente las tres gráficas se aprecian patrones muy distintos que explican el éxito y el fracaso de cada heurística:

- **Mejor Vecino (Línea roja discontinua):** Sufre de un estancamiento casi instantáneo. Al explorar minuciosamente todo el vecindario antes de dar un solo paso, se queda atrapado convergiendo en mínimos locales de muy mala calidad porque no le ha dado tiempo a mejorar (20 millones en la Grande).
- **Algoritmo Genético (Línea verde discontinua):** Tiene bajadas profundas y espectaculares. Gracias al aumento en la probabilidad de mutación (40 %) y la mayor influencia del torneo ($k = 5$), la población es lo suficientemente dinámica como para no atascarse. Logra minimizar el problema grande hasta unos impresionantes 3.6 millones antes de detenerse.

- **Primer Mejor (Línea azul continua):** Sigue siendo la mejor estrategia por excelencia. Su curva describe un descenso prolongado y suave. Al aceptar la primera mejora, exprime de manera muy eficiente cada ciclo de CPU, descendiendo poco a poco hasta el 1.4M.

6.2. Comparativa de Costes y Tiempos

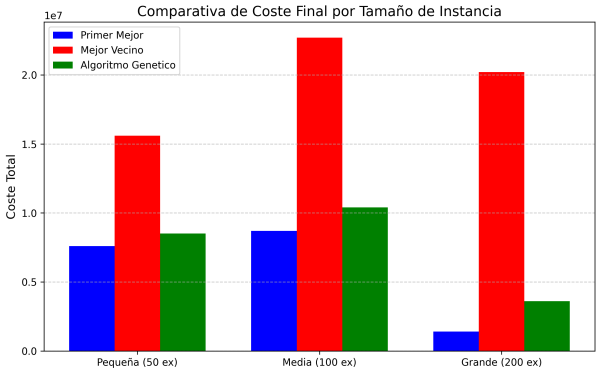

Figura 4: Diagrama de barras de costes finales.

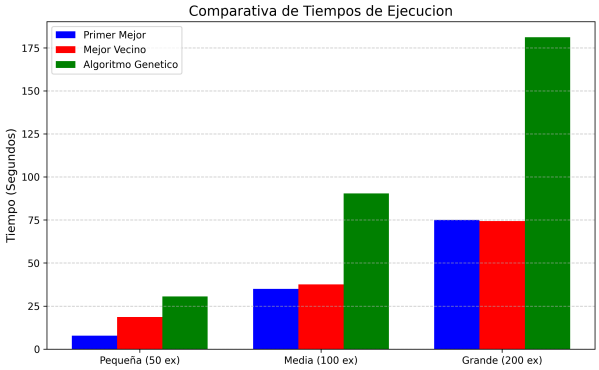

Figura 5: Tiempos de ejecución en CPU.

Si miramos ambas gráficas a la vez, se ve perfectamente lo que comentaba antes y es que el Genético mejora mucho al Mejor Vecino en cuanto al coste final. Sin embargo, la gráfica de la derecha también nos enseña el precio a pagar por esa mejora, que es el tiempo de ejecución, aunque tampoco es una gran diferencia.

Es lógico que el Genético necesite más de 181 segundos en la instancia grande, frente a los 75 segundos en los que se atascan las búsquedas locales. Esto se debe a cómo trabaja cada algoritmo por debajo en la memoria.

En las búsquedas basadas en busquedas locales (Primer Mejor y Mejor Vecino), el programa solo maneja un único individuo (un solo diccionario). Mueve un examen, calcula el nuevo coste y decide si avanza. Es un proceso muy directo y ligero. En cambio, en el Algoritmo Genético, le estoy pidiendo a la CPU que mantenga vivos 60 diccionarios completos y diferentes al mismo tiempo. En cada iteración, el programa tiene que evaluarlos todos, organizar los torneos de selección, crear diccionarios nuevos desde cero cruzando los datos examen por examen, aplicar las mutaciones y gestionar la memoria para reemplazar la población antigua, por todo esto, es lógico que requiera más tiempo de computación.

Aun así, viendo la gran diferencia en la calidad de la solución final, considero que invertir esos 100 segundos extra de computación merece totalmente la pena.

7. Análisis del Impacto del Criterio de Parada

Otro aspecto clave que decidí implementar en el Algoritmo Genético es el criterio de parada por estancamiento. Al hacer pruebas, me di cuenta de que, aunque le había puesto un límite alto de 200 generaciones para darle bastante margen para explorar y evolucionar,

a veces el algoritmo ya había encontrado su mejor solución a mitad de ejecución y se pasaba el resto de las iteraciones calculando para nada, lo cual no era nada eficiente.

Para solucionar esto y hacer que el programa fuera más eficiente con el tiempo de computación, configuré el algoritmo para que el proceso finalice automáticamente si transcurren 20 generaciones consecutivas sin que el fitness del mejor individuo consiga mejorar.

7.1. Justificación de la Mejora

- **Calidad vs. Esfuerzo:** El coste obtenido en la instancia grande (3.6M) aplasta literalmente al de Mejor Vecino (20.2M). Esto confirma que el Genético, cuando se le ajustan bien los parámetros dandole el margen adecuado y se detiene en el momento óptimo de estancamiento de la curva verde, es una metaheurística superior al Mejor Vecino desde mi punto de vista.
- **Gestión de Hardware:** Aunque el Genético invierte más tiempo global, los criterios de parada que he añadido eliminan el cálculo de generaciones finales innecesarioas. Si obligásemos a la CPU a realizar las 200 generaciones completas tras atascarse en la generación 100, el tiempo de ejecución se duplicaría sin ningún propósito real.

sectionAnálisis Empírico de Parámetros del Algoritmo Genético

Para llegar a la configuración óptima final que he mencionado en los apartados anteriores, he realizado un conjunto de experimentos previos evaluando tres parámetros clave en la mayoría de ellos, ya que necesitaba ver como influían en el coste de manera empírica (otros los configuré simplemente intentando entender el completo funcionamiento del algoritmo y la influencia de cada parámetros.

7.2. Tamaño de la Población (N)

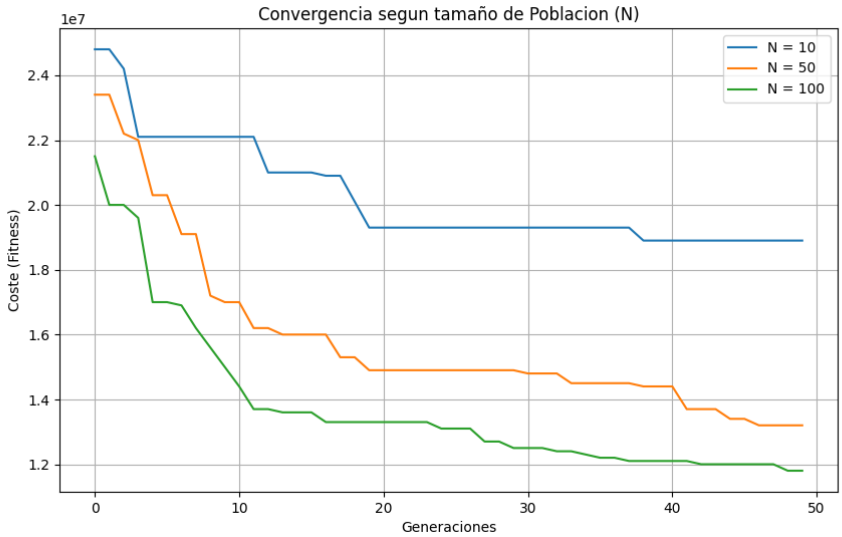

Figura 6: Evolución del fitness según el tamaño de la población (N).

Población (N)	Coste Final	Tiempo de Ejecución (s)
10	16.603.266,32	2,34
50	11.603.350,08	11,70
100	10.603.252,16	23,20

Cuadro 2: Impacto del tamaño de la población.

Análisis: Con $N = 10$, la población es demasiado pequeña, pierde diversidad rápido y se estanca en 16.6 millones. Aumentar a $N = 100$ logra la mejor solución en esta prueba, pero castigaba el tiempo de ejecución. Esta gráfica fue la que motivó mi decisión de fijar una población de $N = 60$ en el modelo final ya que me parece un punto intermedio que aporta suficiente variedad sin penalizar tanto el tiempo, es decir, es muy equilibrado.

7.3. Probabilidad de Cruce (P_c)

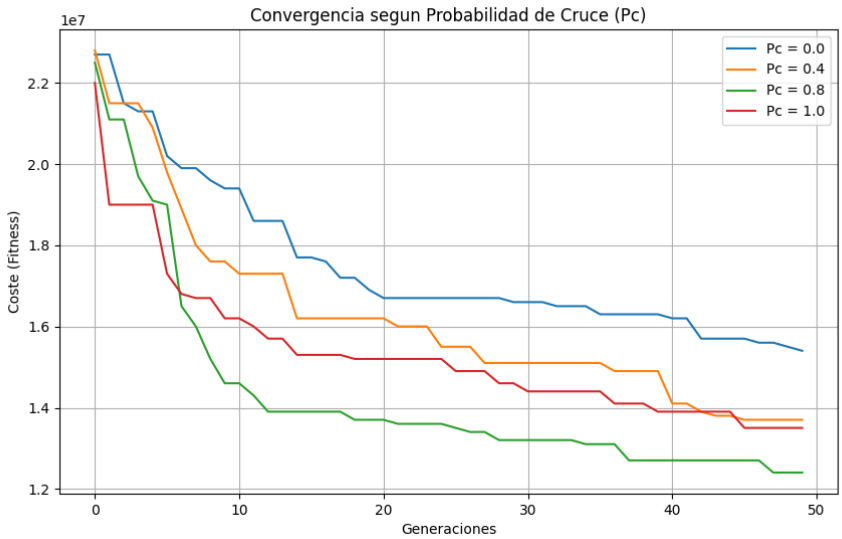

Figura 7: Evolución del fitness según la probabilidad de cruce (P_c).

Prob. de Cruce (P_c)	Coste Final
0.0	16.003.461,40
0.4	11.703.232,16
0.8	12.003.265,16
1.0	12.203.355,32

Cuadro 3: Impacto de la probabilidad de cruce.

Análisis: Estos resultados respaldan la teoría. Al desactivar el cruce ($P_c = 0{,}0$), el coste se dispara hasta los 16 millones, demostrando que la recombinación de "bloques constructivos.^entre

buenos individuos es vital y al final es en lo que se basa el algoritmo. Valores entre 0,4 y 0,8 ofrecían resultados bastante competitivos y bastante útiles para trabajar el algoritmo.

7.4. Probabilidad de Mutación (P_m)

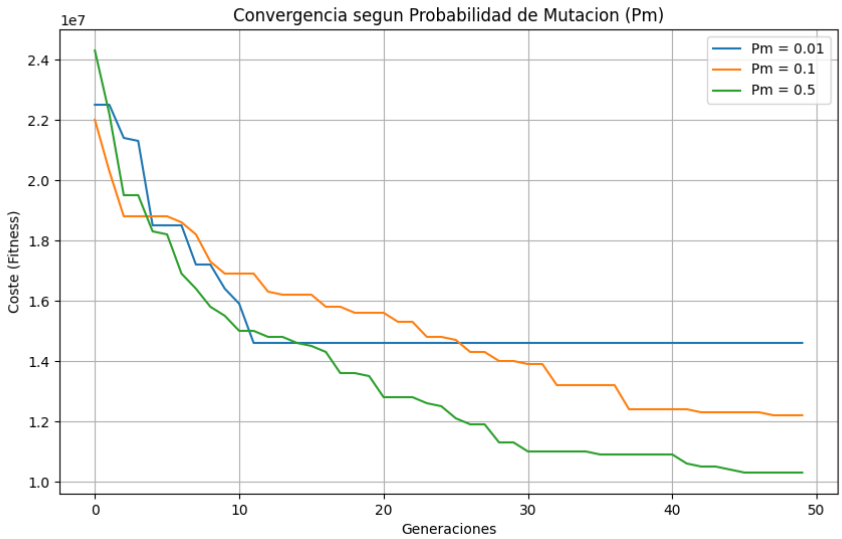

Figura 8: Evolución del fitness según la probabilidad de mutación (P_m).

Prob. de Mutación (P_m)	**Coste Final**
0.01	13.503.290,16
0.10	12.803.273,24
0.50	11.203.186,16

Cuadro 4: Impacto de la probabilidad de mutación.

Análisis: Este fue el descubrimiento más influyente que tuve sobre los resultados de coste del algoritmo. Una mutación muy baja (0,01) penaliza la diversidad y el algoritmo se estanca en 13.5M. Curiosamente, una mutación agresiva del 50 % ($P_m = 0{,}5$) obtuvo el mejor coste. Esto me sugirió que, dado que mi función de mutación no altera gran cantidad de individuos de la población, la probabilidad de aplicación debía ser alta. Apoyado por estos datos, elevé el parámetro definitivo a $P_m = 0{,}4$ buscando nuevamente el equilibrio , siendo este el impulsor principal de la mejora de resultados de mi ejecución final.

8. Conclusiones Finales

Esta segunda práctica ha sido muy importante para permitirme entender mejor la teoría y me ha enseñado practicamente el valor y la importancia del ajuste de hiperparámetros. No

es suficiente con programar un algoritmo inspirado en la naturaleza; hay que entender cómo interactúa con el espacio de búsqueda.

He podido comprobar que una estrategia 'agresiva' como el Primer Mejor es buena por su rapidez, pero que un Algoritmo Genético, cuando se configuran empíricamente su selectividad ($k = 5$) y su tasa exploratoria ($P_m = 0{,}4$), se convierte en una maquinaria capaz de mejorar los costes de millones de penalizaciones a base de evolución mejorando los genes en cada iteración.

9. Repositorio del Código

El código fuente completo implementado para esta práctica, así como las instrucciones detalladas de ejecución, las he subido a un repositorio público de Git. Se puede acceder a él a través del siguiente enlace: `https://github.com/ilias04ahmed/MH_Practica2`